



Digital IC Design & Verification

Physical Design

Lab Manual # 04 | IO Pads Placement

Release: 1.0

Date: 15-September-2025

NUST Chip Design Centre (NCDC), Islamabad, Pakistan



Copyrights ©, NUST Chip Design Centre (NCDC). All Rights Reserved. This document is prepared by NCDC and is for intended recipients only. It is not allowed to copy, modify, distribute or share, in part or full, without the consent of NCDC officials.

Revision History

Revision Number	Revision Date	Revision By	Nature of Revision	Approved By
1.0	15/09/2025	Fawad Ahmed / Muhammad Junaid	Complete manual	Sharjeel Khilji
1.1	08/06/2026	Fawad Ahmed	Conversion into Command-Base	-



Table of Contents

Objective.....	3
Tools.....	3
Instructions.....	3
1. Introduction.....	4
2. Learning Objectives	4
3. Pads Design.....	5
4. Pads Arrangements.....	7
5. Pads Placement.....	12
6. I/O file for Innovus.....	14
7. Constraint file (.sdc).....	18
8. Synthesis.....	19
9. Placement & Route	19
10. Lab Task	26
11. References	26



Objective

The objective of this lab is to

- Place IO Standard Cells of desired library in your desired RTL design.
- Understand the IO placement locations in IO file.
- Pre/Post Simulate the IO Pads based RTL Design to verify the functionality.
- Final STA after layout to verify whether there is timing violation or not.

Tools

- Cadence Xcelium
- Cadence Genus
- Cadence Innovus
- Cadence Tempus

Instructions

The students are required to design and submit the following files for evaluation of this lab which is given below:

- The report should be in detail. Each steps & commands should be explained properly. Also, the results & its effects should be explained.
- Your files should be submitted in this format i.e.
 - Name – Module – Lab Number.zip [Includes all required sv, rpt, sdc, tcl files]
 - Name – Module – Lab Number.pdf [Your Report]



1. Introduction

In modern integrated circuit (IC) design, physical implementation is a critical stage where the abstract representation of a design is transformed into a manufacturable layout. One of the most important steps in this process is the **placement of Input/Output (IO) pads**. IO pads serve as the physical interface between the internal core logic of the chip and the external environment, enabling communication with other chips, packages, and the printed circuit board (PCB).

Proper IO pad placement is essential for:

- Ensuring **signal integrity** by minimizing noise and cross-talk.
- Supporting **reliable power delivery** through dedicated VDD and VSS pads distributed around the periphery.
- Meeting **electrostatic discharge (ESD) protection** requirements.
- Achieving correct **bonding or bump connections** in packaging.
- Maintaining an optimized **floorplan** that balances core utilization, power routing, and timing closure.

This lab focuses on learning the **principles and methodology of IO pad placement** in ASIC physical design. Through hands-on steps, you will explore how to:

- Define the die and core area.
- Allocate space for power rings and routing margins.
- Place IO pads systematically around the die.
- Connect IO pads to the core logic for simulation and physical verification.

2. Learning Objectives

After completing this lab on **IO Pads Placement in Physical Design**, you will be able to:

1. **Understand the purpose of IO pads** in ASIC design, particularly their role in connecting internal core logic to external package pins.
2. **Differentiate between signal pads and corner pads**, and explain why they are essential in chip-level design.
3. **Define the die outline and core dimensions**, and allocate space for IO pad rows around the periphery.
4. **Perform systematic placement of IO pads** around the die boundary to form a pad ring.
5. **Apply placement rules** to maintain uniform distribution of IO pads along each side of the die.
6. **Verify pad-to-core signal connectivity** using placement and netlist checks.
7. **Analyze floorplan utilization** to ensure that pad placement does not affect core placement or routing.



3. Pads Design

In VLSI physical design, the overall **chip size** is determined either by the **logic core area** or by the **I/O pad requirements**. The relationship between these two factors classifies a design as either **core-limited** or **pad-limited**.

3.1. Core-Limited Design

A design is called **core-limited** when the **logic area** (standard cells, macros, memory blocks, etc.) is the dominating factor in determining chip size. In this case, the core occupies most of the silicon area, and the I/O pads can be easily accommodated around the periphery without enlarging the die.

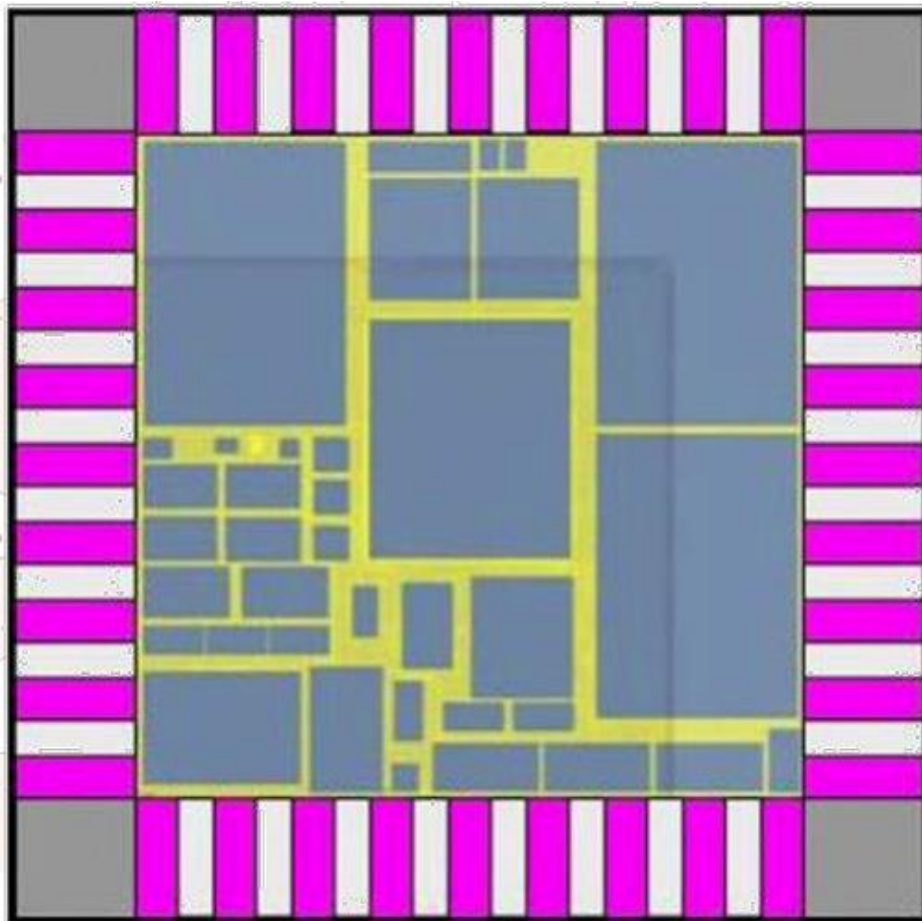


Figure 1: Core Limited Design

- **Characteristics:**

- Large logic area compared to the number of pads.
- Pads occupy only a small portion of the chip perimeter.
- The die edge length is approximately determined by:

$$L_{chip} \approx L_{core}$$

- Typical for **high-complexity digital chips** (e.g., processors, GPUs, SoCs)



- **Advantages:**
 - Better utilization of silicon.
 - Pads do not waste much area.
- **Disadvantages:**
 - Requires careful floorplanning to route signals efficiently inside a large core.

3.2. Pad-Limited Design

A design is **pad-limited** when the **number of I/O pads** (including signal, power, and ground pads) dominates chip size. Even though the logic area (core) is small, the large number of pads requires more periphery, thereby increasing chip dimensions.

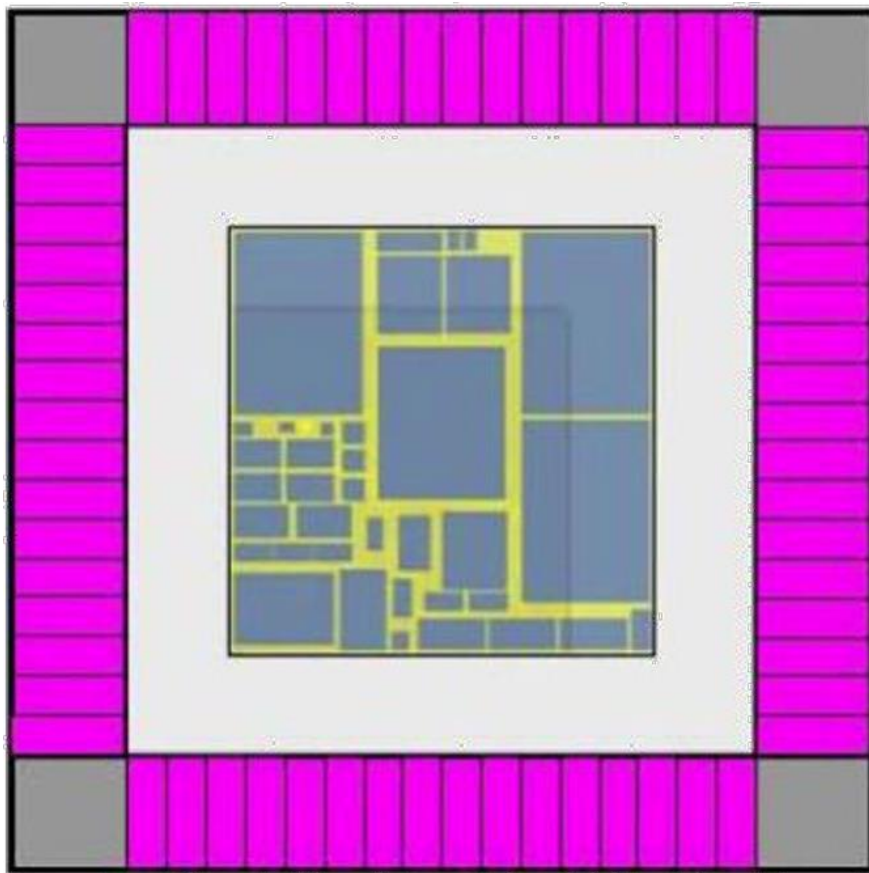


Figure 2: Pad-Limited Design

- **Characteristics:**
 - Small logic core compared to many I/O pads.
 - Chip size is determined by pad spacing, not logic area.
 - The die edge length is approximately:
$$L_{\text{chip}} \approx L_{\text{pad}} = (N_{\text{pads}} * P_{\text{pad}}) / 4$$
where N_{pads} = total number of pads and P_{pad} = pad pitch (center-to-center spacing between two adjacent [Pad Width + Pad Spacing]).



- Typical for **simple designs with large I/O requirements**, such as GPIO controllers, small counters, and test chips.
- **Advantages:**
 - Easier to route I/O signals because of larger pad ring.
- **Disadvantages:**
 - Poor silicon utilization → a large fraction of chip area remains unused.
 - Higher cost per die due to wasted silicon.

Here is the table of comparison of Pad Design which is given below:

Aspect	Core-Limited	Pad-Limited
Chip size decided by	Core logic area (Std.Cells Logic)	Pad requirements (# of I/O Pads & Spacing)
Silicon utilization	High (core fills most of chip)	Low (large empty area inside pad ring)
Seen in	CPUs, DSPs, SoCs	Small logic + many I/Os (e.g., counters)
Example metric	$L_{core} > L_{pad}$	$L_{pad} > L_{core}$

4. Pads Arrangements

When designing the pad ring for an integrated circuit (IC), the arrangement of pads around the die periphery is an important consideration. The way pads are placed determines how bond wires or solder bumps will connect the chip to the package. Two commonly used pad configurations are **In-Line** and **Staggered**.

4.1. In-Line Pads

In this scheme, all the IO pads are placed **next to each other in a single row** along the edge of the die. The corresponding **bond pads** (the metal landing points for wire bonding) are lined up directly opposite one another, creating a neat and uniform bond pad array. A small gap is usually left between adjacent pads to ensure manufacturability and to avoid short circuits. The total number of pads per edge is limited by chip edge length:

$$N_{pads/edge} = P_{pad} / L_{edge}$$

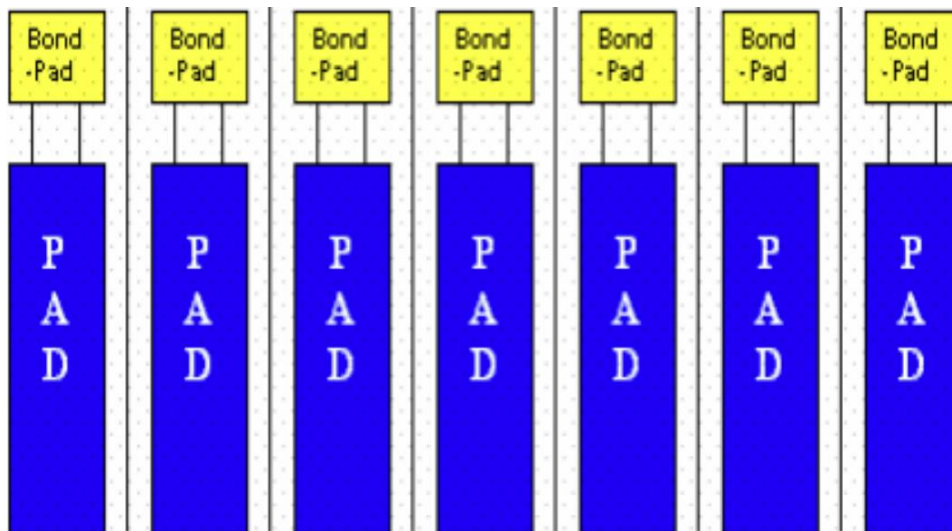


Figure 3: In-Line Pad Configuration

- **Advantages:**
 - Simple to design and easy to implement.
 - Best suited when the number of pads is small to moderate.
 - Common in **small pad-limited chips** and student projects.
 - Maintains compact die height since only one row is used.
- **Disadvantages:**
 - Limited pad density. If the design requires many IO signals, the die size may need to be increased just to fit all pads along the periphery.
 - May result in “pad-limited” designs, where pad placement dictates die size rather than core logic.

4.2. Staggered Pads

In this scheme, IO pads are placed in **two offset rows** along the edge of the die (like a zig-zag or staggered arrangement). One row is closer to the die edge (outer pads), while the other is slightly inside (inner pads). Bond pads are also staggered in two rows, enabling more pads to fit in the same length of die edge.

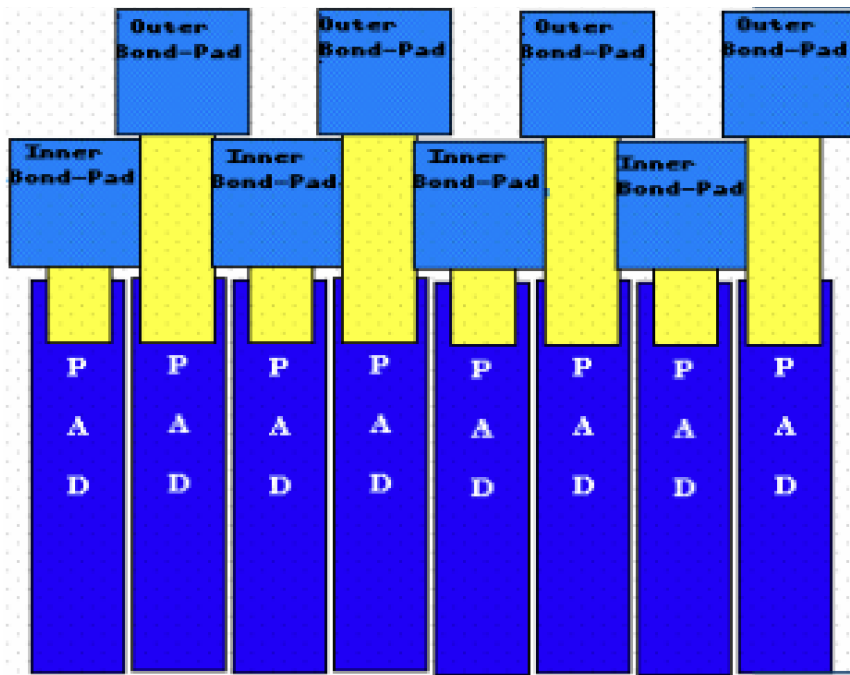


Figure 4: Staggered Pad Configuration (non-CUP)

- **Advantages:**
 - Useful for designs that are **pad-limited**, since more pads can be accommodated without increasing die width.
 - Increases pad density significantly compared to In-Line configuration.
- **Disadvantages:**
 - The overall pad ring height increases, since two rows are required.
 - Slightly more complex in terms of routing signals from the core to inner-row pads.
 - Wire bonding becomes more complex because bond wires for inner pads must cross over outer ones without interference.

4.3. Circuit Under Pads (CUP)

The **Circuit Under Pad (CUP)** methodology is an advanced IO pad design technique used in modern integrated circuits (ICs) to save die area and improve layout efficiency. Unlike conventional IO placement where the bonding pad and IO circuitry are placed side-by-side, in CUP the bond pad is positioned **directly above** the IO circuitry.

Key Points

- **Bond Pad over IO Body**
 - In CUP design, the large bonding pad (where wire bonding or flip-chip bump connection occurs) is placed **on top of the IO circuit cell itself**.
 - This differs from traditional layouts, where the bonding pad occupies its own dedicated silicon area separate from the IO circuitry.



- **Electrical Connectivity**
 - The bonding pad is electrically connected to the IO circuit by routing the signal to the **PAD pin** of the IO cell.
 - Even though the pad sits physically above the circuitry, proper design ensures there is no electrical short or reliability issue. Special passivation layers and shielding techniques are used to protect the underlying transistors.
- **Die Size Reduction**
 - CUP is widely adopted in pad-limited designs where the periphery area occupied by pads dictates the die size.
 - By placing the bonding pad directly above the IO circuit, **no additional silicon area** is required for the pad.
 - This allows for **significant reduction in die size**, leading to cost savings and improved silicon utilization.
- **Advantages of CUP**
 - Substantially reduces die area in pad-limited designs.
 - Enables higher pad density without requiring staggered pad rows.
 - Allows more efficient floorplanning and utilization of chip periphery.
- **Challenges of CUP**
 - Requires careful design of **passivation openings** to ensure mechanical reliability of the bond pad.
 - The underlying IO circuitry must be designed with sufficient **robustness against stress** from bonding.
 - Adds complexity to manufacturing and verification steps.

In this lab, we will use TSMC 65nm IO library named in which files are given below:

- **Lef:** *tpzn65lpgv2od3_9lm.lef*
The file name breakdown contains:
 - **tpzn65** → Refers to **TSMC 65nm Process Node** (low-power CMOS library).
 - **lp** → Low-Power (optimized for leakage and power).
 - **gv2od3** → Naming specific to the IO/Pad library variant (it often encodes voltage domain, drive strength, and ESD type).
 - **9lm** → This process supports **9 Metal Layers** for routing.So, this LEF contains the abstract layout of IO pad cells for 65nm low-power technology with 9 routing layers.
- **Libs:** *tpzn65lpgv2od3bc.lib | tzn65lpgv2od3wc.lib | tpzn65lpgv2od3tc.lib*
- **Verilog:** *tpzn65lpgv2od3.v*



The Standard Cells of Pad which we will use are **PDDW16DGZ** for data I/Os and reset I/P, & **PDXOE3DG** for clock I/P. The circuits & truth tables for these cells are given below:

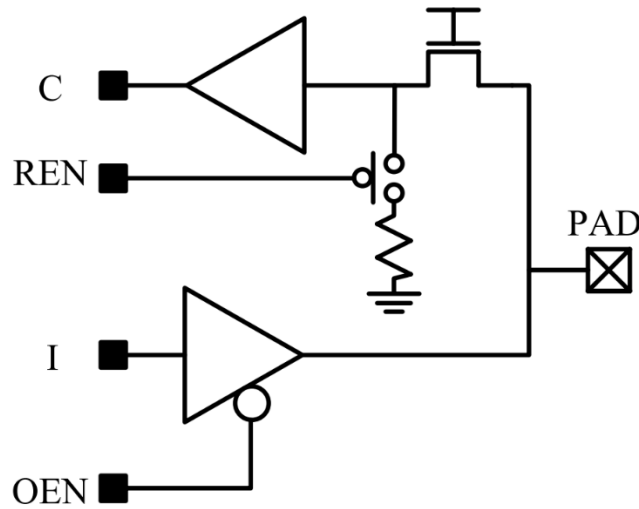


Figure 5: PDDW16DGZ | Tri-State Output Pad with Input and Enable Controlled Pull-Down

Truth Table

INPUT				OUTPUT	
OEN	I	PAD	REN	PAD	C
0	0	-	0/1	0	0
0	1	-	0/1	1	1
1	0/1	0	0/1	-	0
1	0/1	1	0/1	-	1
1	0/1	Z	0	-	0
1	0/1	Z	1	-	X

Figure 6: Truth Table

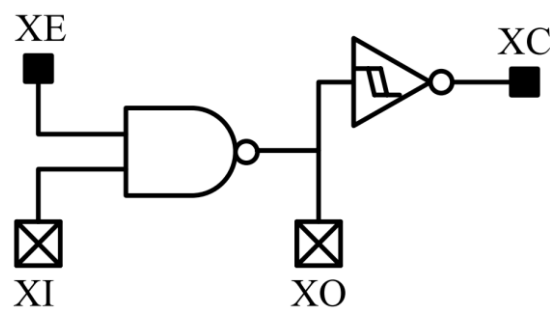


Figure 7: PDXOE3DG | Crystal Oscillator Cell (High Enable)



Truth Table			
INPUT		OUTPUT	
XE	XI	XO	XC
0	0/1	1	0
1	0	1	0
1	1	0	1

Figure 8: Truth Table

Each I/Os of desired RTL design should be instantiated with these pads I/Os which will be explained in next section.

5. Pads Placement

For this step, we will use our 8-Bit ALU design operated by clock which RTL is given below:

```
1 `timescale 1ns/10ps
2 module ALU(Out, A, B, Sel, clk, rst);
3     input [7:0] A, B;
4     input [1:0] Sel;
5     input clk, rst;
6     output reg [7:0] Out;
7
8     wire [7:0] M1, A1, N1, N2;
9     reg [7:0] M_Out;
10
11     assign M1 = A & B;    // AND
12     assign A1 = A | B;    // OR
13     assign N1 = ~A;       // NOT A
14     assign N2 = ~B;       // NOT B
15
16     always @(posedge clk) begin
17         if (!rst)
18             Out <= 8'b0;
19         else
20             Out <= M_Out;
21     end
22
23     always @(*) begin
24         case (Sel)
25             2'b00 : M_Out = M1;
26             2'b01 : M_Out = A1;
27             2'b10 : M_Out = N1;
28             2'b11 : M_Out = N2;
29         endcase
30     end
31 endmodule
```

Figure 9: RTL of 8-Bit ALU

Now we will instantiate the IO Cells with our desired RTL design which is given below:



```
1 module top_wrapper(Out, A, B, Sel, clk, rst);
2     input [7:0] A, B;
3     input [1:0] Sel;
4     input clk, rst;
5     output [7:0] Out;
6
7     // Internal nets
8     wire [7:0] wA, wB, wOut;
9     wire [1:0] wSel;
10    wire wclk, wrst;
11
12    // Core logic
13    ALU uut (
14        .A(wA),
15        .B(wB),
16        .Sel(wSel),
17        .clk(wclk),
18        .rst(wrst),
19        .Out(wOut)
20    );
21
22    // IO pads for signals
23    PDX0E3DG clk_inst (.E(1'b1), .XIN(clk), .XC(wclk), .XOUT());
24    PDDW16DGZ rst_inst (.OEN(1'b1), .I(1'b1), .PAD(rst), .REN(1'b1), .C(wrst));
25
26    // A inputs
27    PDDW16DGZ A_inst0 (.OEN(1'b1), .I(1'b1), .PAD(A[0]), .REN(1'b1), .C(wA[0]));
```

Figure 10: Sample Top Module

```
26 // A inputs
27 PDDW16DGZ A_inst0 (.OEN(1'b1), .I(1'b1), .PAD(A[0]), .REN(1'b1), .C(wA[0]));
28 PDDW16DGZ A_inst1 (.OEN(1'b1), .I(1'b1), .PAD(A[1]), .REN(1'b1), .C(wA[1]));
29 PDDW16DGZ A_inst2 (.OEN(1'b1), .I(1'b1), .PAD(A[2]), .REN(1'b1), .C(wA[2]));
30 PDDW16DGZ A_inst3 (.OEN(1'b1), .I(1'b1), .PAD(A[3]), .REN(1'b1), .C(wA[3]));
31 PDDW16DGZ A_inst4 (.OEN(1'b1), .I(1'b1), .PAD(A[4]), .REN(1'b1), .C(wA[4]));
32 PDDW16DGZ A_inst5 (.OEN(1'b1), .I(1'b1), .PAD(A[5]), .REN(1'b1), .C(wA[5]));
33 PDDW16DGZ A_inst6 (.OEN(1'b1), .I(1'b1), .PAD(A[6]), .REN(1'b1), .C(wA[6]));
34 PDDW16DGZ A_inst7 (.OEN(1'b1), .I(1'b1), .PAD(A[7]), .REN(1'b1), .C(wA[7]));
```

Figure 11: I/P Pads Group A

1. PXOE3CDG clk_pad (.E (1'b1), .XIN(clk_in), .XOUT(), .XC(wclk));

In this cell:

- We fed external clock input into XIN.
- E is set as high that enables clock buffer.
- Then we get outputs i.e XOUT which set as undeclared as we no need to use that.
- XC that will feed to the core (i.e. ALU).

2. PDDW16DGZ rst_inst (.OEN(1'b1), .I(1'b1), .PAD(rst), .REN(1'b1), .C(wrst)); In this cell:

- We fed external reset into PAD and went out from C which is set as wrst.
- I set as high that disables O/P buffer (Core to Pad Transmission)
- REN set as high that I/P when no external driver is connected.
- OEN set as high that disable O/P so pad works as I/P.

3. PDDW16DGZ A_inst0 (.OEN(1'b1), .I(1'b1), .PAD(A[0]), .REN(1'b1), .C(wA[0]));

In this cell:

- We fed external A[0] into PAD and went out from C which is set as wA[0].



- I set as high that disables O/P buffer (Core to Pad Transmission)
- REN set as high that I/P when no external driver is connected.
- OEN set as high that disable O/P so pad works as I/P.

Same goes to all I/P of A[7:0], B[7:0] & Sel[1:0].

```
50 // Output pads
51 PDDW16DGZ out_inst0 (.OEN(1'b0), .I(wOut[0]), .PAD(Out[0]), .REN(), .C());
52 PDDW16DGZ out_inst1 (.OEN(1'b0), .I(wOut[1]), .PAD(Out[1]), .REN(), .C());
53 PDDW16DGZ out_inst2 (.OEN(1'b0), .I(wOut[2]), .PAD(Out[2]), .REN(), .C());
54 PDDW16DGZ out_inst3 (.OEN(1'b0), .I(wOut[3]), .PAD(Out[3]), .REN(), .C());
55 PDDW16DGZ out_inst4 (.OEN(1'b0), .I(wOut[4]), .PAD(Out[4]), .REN(), .C());
56 PDDW16DGZ out_inst5 (.OEN(1'b0), .I(wOut[5]), .PAD(Out[5]), .REN(), .C());
57 PDDW16DGZ out_inst6 (.OEN(1'b0), .I(wOut[6]), .PAD(Out[6]), .REN(), .C());
58 PDDW16DGZ out_inst7 (.OEN(1'b0), .I(wOut[7]), .PAD(Out[7]), .REN(), .C());
```

Figure 12: Output Pads

4. PDDW16DGZ out_inst0 (.OEN(1'b0), .I(wOut[0]), .PAD(Out[0]), .REN(), .C());

In this cell:

- We fed internal wOut[0] into I and went out from PAD which is set as Out[0].
- Pad works as O/P, so no need to use REN & C.
- OEN set as low that enable O/P so pad works as O/P.

Similarly goes to all output pins [7:0].

For more details, you can refer the datasheet which is uploaded on LMS. You should instantiate these core I/O pins (of any RTL design) to I/O Pad cells (technology specified library) on your own.

6. I/O file for Innovus

In ASIC physical design, once the floorplan of the chip is created, the next major task is to define how I/O pads (input, output, power, and corner pads) are placed around the chip boundary.

Why is this needed?

- The core logic (like your ALU) sits in the center of the chip.
- Signals from the outside world (pins, package, PCB) must pass through pads to reach the core.
- Pads provide electrical buffering, ESD protection, and proper drive strength for signals.

To tell Innovus (the PnR tool) where to place these pads, we provide an IO file.

5.3 Types of Pads

1. Corner Cells

- These are special cells used at the four corners of the die.
- They ensure the power ring continuity (VDD/VSS) and define the geometry of the pad frame.
- Without them, the pad ring would remain "open" at corners.



2. Input/Output Pads

- These pads connect functional signals of your design (for example: A[7:0], B[7:0], Sel[1:0], clk, rst, Out[7:0]).
- Each input or output of your RTL must have a corresponding IO pad at the boundary.

3. Power/Ground Pads

- These pads bring VDD (power) and VSS (ground) into the chip.
- They are connected to the power ring inside the chip to supply current to all cells.

5.3 Why I/O File Needed?

The IO file is like a blueprint that defines:

- Which pads are used (their cell type).
- Where they are placed (top, bottom, left, right).
- In what order they appear along each edge.
- Orientation of each pad (so that pins face outwards).
- Placement status (fixed so Innovus won't move them).

Without an IO file, Innovus only knows you have pad instances in the netlist, but it doesn't know how to arrange them around the chip.

Imagine building a house:

- Floorplan = placing the rooms.
- IO file = telling where to put doors and windows (the only way to interact with the outside world).

If you skip it, the tool may place pads randomly, leading to incorrect pin ordering that won't match your package/PCB connections.

5.3 I/O File Structure

1. Globals Section:

```
(globals
  version = 3
  io_order = clockwise
  space = 10
)
```

Figure 13: Globals in I/O File

- **version = 3** → IO file format version used by Innovus.
- **io_order = clockwise** → Pads placed clockwise around the core (from top-left corner).
- **space = 10** → Minimum spacing between pads in microns.



2. I/O Pad Section:

This section specifies pad placement on each side of the die.

i. Corner Pads

```
(topleft
  (inst name="CornerCell1" cell=PCORNER orientation=R270 place_status=fixed)
)
```

Figure 14: Top Left Side

- **Inst name = "Corner Cell1"** → Name for each cells
- **Cell = PCORNER** → corner pad cell from technology library.
- **Orientation = R270** → rotates the cell properly (90° steps) for each corner.
- **place_status = fixed** → tool cannot move it, locked in place.

You repeat for 4 corners [top-left (Rotation 270°), top-right (Rotation 180°), bottom-right (Rotation 90°), bottom-left (Rotation 0°)].

ii. Left Edge (A[7:0] + Reset)

```
; A[7:0] + Reset
(left
  (inst name="A_inst0" cell=PDDW16DGZ place_status=fixed)
  ...
  (inst name="rst_inst" cell=PDDW16DGZ place_status=fixed)
)
```

Figure 15: Left Side

- **left** → pads placed on the left side of the die. To place I/O cells other than left, you can write keywords **right**, **top** & **bottom**.
- **inst name = "A_inst0"** → instance name matching the Verilog pad.
- **Cell = PDDW16DGZ** → pad cell type from the library.
- **place_status = fixed** → pad is fixed in position (not movable).

Here, the left edge includes:

- All A[7:0] input pads (data inputs).
- The reset pad.

Each pad is fixed in order (bottom to top along left edge).

iii. Top Edge (B[7:0] + Sel[1:0])



```
; B[7:0] + Sel[1:0]
(top
  (inst name="B_inst7" cell=PDDW16DGZ place_status=fixed)
  ...
  (inst name="S_0_inst" cell=PDDW16DGZ place_status=fixed)
)
```

Figure 16: Top Side

Here, top edge includes:

- All **B[7:0]** inputs.
- **Sel[1:0]** pads (control inputs).

Ordered left-to-right along the top edge.

iv. Right Edge (Out[7:0])

```
; Out[7:0]
(right
  (inst name="out_inst0" cell=PDDW16DGZ place_status=fixed)
  ...
)
```

Figure 17: Right Side

Here, right edge includes all **Out[7:0]** outputs.

Ordered top-to-bottom along the right edge.

v. Bottom Edge (Out[7:0])

```
; Clock
(bottom
  (inst name="PVSSB" cell=PVSS1DGZ place_status=fixed)
  (inst name="clk_inst" cell=PDXOE3DG place_status=fixed)
  (inst name="PVddb" cell=PVDD1DGZ place_status=fixed)
)
```

Figure 18: Bottom Side

Here, bottom edge includes **clock input**.

You can make this file on your own according to your design, this is sample file that describes its structure. You should also add power pads (named as PVDD1DGZ & PVSS1DGZ) on each side of die i.e VDDL & VSSL for left edge, VDDT & VSST for right edge and so on. Make sure that the VDD & VSS cells must not place at edge of adjacent sides of die as it will get DRC violations when routing metal between power pads with power rings of core. Also make sure that each instance name in IO file is matched with instance name written in RTL (which you have instantiated the IO cells with each I/Ps & O/Ps). You can



modify these IO Pads locations in Innovus GUI also which will be describe later.
For more details, you can refer this [IO File](#) video.

```
1 (globals
2   version = 3
3   io_order = clockwise
4   space = 10
5 )
6
7 (iopad
8   ;===== Corner Pads =====
9   (topleft
10    (inst name="CornerCell1" cell=PCORNER orientation=R270 place_status=fixed)
11   )
12   (topright
13    (inst name="CornerCell2" cell=PCORNER orientation=R180 place_status=fixed)
14   )
15   (bottomright
16    (inst name="CornerCell3" cell=PCORNER orientation=R90 place_status=fixed)
17   )
18   (bottomleft
19    (inst name="CornerCell4" cell=PCORNER orientation=R0 place_status=fixed)
20   )
21
22   ;===== Left Edge =====
23   ; A[7:0] + Reset
24   (left
25    (inst name="A_inst0" cell=PDDW16DGZ place_status=fixed)
26    (inst name="A_inst1" cell=PDDW16DGZ place_status=fixed)
27    (inst name="A_inst2" cell=PDDW16DGZ place_status=fixed)
28    (inst name="PVDDL" cell=PVDD1DGZ place_status=fixed)
29    (inst name="A_inst3" cell=PDDW16DGZ place_status=fixed)
30    (inst name="A_inst4" cell=PDDW16DGZ place_status=fixed)
31    (inst name="A_inst5" cell=PDDW16DGZ place_status=fixed)
32    (inst name="PVSSL" cell=PVSS1DGZ place_status=fixed)
33    (inst name="A_inst6" cell=PDDW16DGZ place_status=fixed)
34    (inst name="A_inst7" cell=PDDW16DGZ place_status=fixed)
35    (inst name="rst_inst" cell=PDDW16DGZ place_status=fixed)
36   )
37 )
```

Figure 19: Complete Structure of I/O File

7. Constraint file (.sdc)

In previous labs, you have already written the .sdc file for different designs without using IO Pads but in this case, few keywords are change which is descried below:

1. Clock Definition

```
create clock -name CLK -period 10 -waveform {0 5} [get_pins {clk_inst/XC}]
```

In your previous labs, you have used **get_ports** that refers to top-level design ports (chip-level inputs/outputs) but in this case you should use **get_pins** that refers to internal pins of cells/instances inside the netlist. We written as `get_pins {clk_inst/XC}` where `clk_inst` is instance name (written in RTL) and `XC` is output that moves towards the core.



2. Input Constraints

```
set_input_delay -clock CLK -max 2.0 [get_pins {A_inst0/C A_inst1/C A_inst2/C A_inst3/C A_inst4/C A_inst5/C A_inst6/C A_inst7/C}]  
set_input_delay -clock CLK -min 0.5 [get_pins {A_inst0/C A_inst1/C A_inst2/C A_inst3/C A_inst4/C A_inst5/C A_inst6/C A_inst7/C}]
```

In this statement, we use `get_pins {A_inst[*]/C}` where `A_inst[*]` is instance name and `C` is output that moves towards core.

3. Output Constraints

```
set_output_delay -clock CLK -max 5.0 [get_pins {out_inst0/I out_inst1/I out_inst2/I out_inst3/I out_inst4/I out_inst5/I out_inst6/I out_inst7/I}]  
set_output_delay -clock CLK -min 2.0 [get_pins {out_inst0/I out_inst1/I out_inst2/I out_inst3/I out_inst4/I out_inst5/I out_inst6/I out_inst7/I}]
```

In this statement, we use `get_pins {out_inst [*]/I}` where `out_inst [*]` is instance name and `I` is input that takes O/P from core.

The full version of this sdc file is attached in LMS. You can refer this file & understand each statement & its purpose. Now we will begin our front-end & back-end process of this design.

8. Synthesis

We will use 8-Bit ALU for front-end & back-end process of Digital IC Design. For this purpose, we will use Genus to synthesize & obtain technology mapped gate-level netlist. To initiate this process, we need the following files which is given below:

- *script.tcl* (tool command language that will guide the Genus to run desired design)
- *constraint.sdc* (defines how the tool should understand timing, clocks, IO delays, loads, and uncertainties)
- *tpzn65lpgv2od3tc.lib*
- *tcbn65lptc.lib*
- *ALU_8B.v* (Main RTL Design)
- *Top_Wrapper.sv* (Instantiation with RTL design & I/O Pads Cells)

We will not run through all front-end process as you have already done in your previous labs. After synthesizing, you will get netlist of your ALU design. Kindly review your reports that are generated by genus tool & make decisions according to these reports. Also perform post simulate your synthesized design to check functionality. Now we move towards the back-end process of DIC Design.

9. Placement & Route

After obtaining the netlist, we will begin back-end process of this design. So, in order to do that we will need the following files:

- *Top_Wrapper_Netlist.v*
- *tcbn65lp_9lmT2.lef*
- *tpzn65lpgv2od3_9lm.lef*
- *IOfile.io*



- *MMMCfile.view*

Although you have done the backend process in your previous labs but some process will explain which is given below:

1. Initializing the design

As you observed in the given script that these files are placed in assigned options but one important thing to make sure that the lef files should be placed in this order (i.e. ***tcbn65lp_9lmT2.lef*** placed 1st, then ***tpzn65lpgv2od3_9lm.lef***), wrong order will get error while initializing the design. In **MMMC.view** file, you have already done in your previous labs. Instead of giving files of simple standard cell library for best-case, worst-case & typical-case, you should add three more files that includes I/O cells library for these cases.

After initializing the design, we will get output like this below:



Figure 20: First Layout

As you observed that the I/O pads are placed according to the I/O file that describes the coordinates of each cell.

2. Floorplanning

```
set_io_flow_flag 1

# Initialize Floorplan boundaries and site rows
create_floorplan -core_size {350.0 350.0 20.0 20.0 20.0 20.0} \
                 -core_margins_by io \
                 -floorplan_origin center \
                 -site core \
                 -match_to_site \
                 -flip s
```

Figure 21: Floorplan Command



In this step, we will explain each required parameter which is given below:

- The core size by dimension is **15um** height & **15um** width which obtained from the area report that includes standard cells area. The calculation is:
 - **Core area with 70% utilization** = Std.Area / 0.7
 - **Width/Height** = Sq.Root(Core Area) = **approx. 15um**

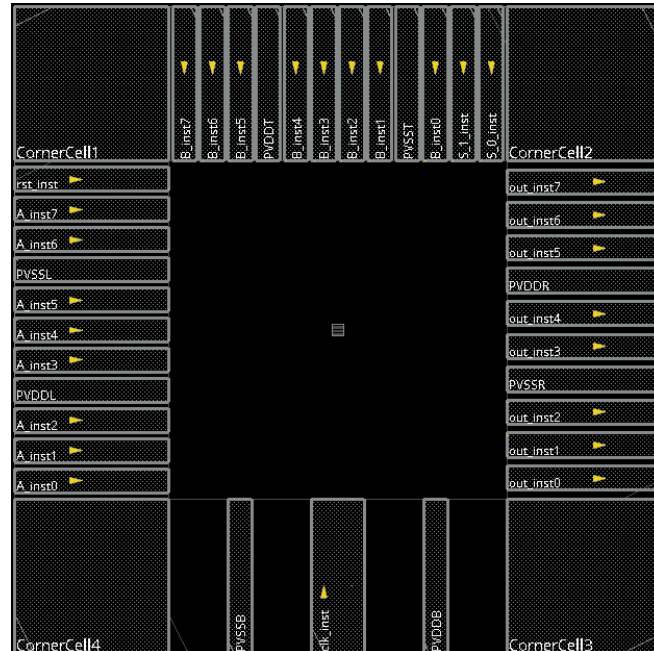


Figure 22: Layout with Smaller Core Area

In the IO pads, we can't decrease the size of die due to size of I/O Pads even decreasing the size of core. If we decrease the size of core and also power rings are placed around the core & far away from I/O Pads, the layout becomes messy when placing metals. Moreover, the tool will not route to connect the power pads with power rings of core. So, in order to mitigate that, we need to increase the core area even the design is smaller. In our case, we put **300um** in height & width but make sure that we need some room left to place power rings of core.

- **Core Margins by Core to IO Boundary** is **310um** of each side as this margin is left for power rings, routing channels, decoupling capacitors, and blockages. The standard formula for this case is:

Core-to-IO Margin $\geq$ IO Pad Height + Width of Power rings + Routing margin

- **IO Pad Height** is obtained from lef file. All cells have same height but width is change. Here in this case, the height is **190 um**.
- **Width of power ring** consists of VDD + VSS rings around the core. Each ring has metal width + spacing. For this design, we put value of metal width is **5.4um** while spacing is **3.6um** let's say, therefore, the width of power rings is **2 Metals * (Width of each metal + Spacing)** which is equal to **18um**.
- **Routing Margin** is the extra space kept between the core boundary and the I/O boundary (pad ring / die edge) so that power rings can



be placed, Clock & power distribution networks (PDN) fit properly, and Global routing has room before entering the core area. The standard value for TSMC 65 is **100-200um**.

- Therefore, the Core to IO Margin is **308um** approx. **310um**.

Remember, this value can be different depending on your design. So, calculate accordingly. In our case, we put **20um** in Core to I/O Margin, you should try above value which we have calculated.

- **Die size calculation** uses *Max IO Height*. This ensures IO pads fit properly around the die boundary.
- **Floorplan origin** set at *Center* as it is easy to visualize balance around the origin. You can also try to set floorplan origin at *lower left corner* & observe the results.
- **Use IO Rows for IO Placement** is set as *High* that ensures IO cells (pads) are placed along pad rows, not random positions.
- **Adjust to site** is set as *High* that aligns IOs to legal sites.

After giving these parameters, we will get desired output which is given below:

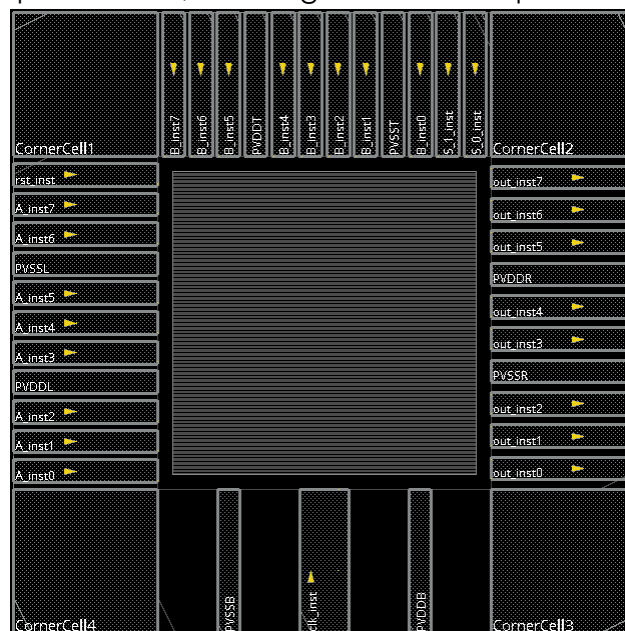


Figure 20: Layout After Floorplanning

3. Power Planning

- **Power Rings & Stripes**

After applying the required floorplanning parameters, the design transitions into the power planning phase. As a preliminary step in this process, the `route_special` command is utilized to route the Pad Ring structure, specifically:

```
# Route the Pad Ring structure
route_special -connect {pad_ring} -nets "$design(digital_gnd) $design(digital_vdd)"
```

Figure 24: SRoute Command



The layout is look like this after applying route alongwith power rings commands which is shown below:

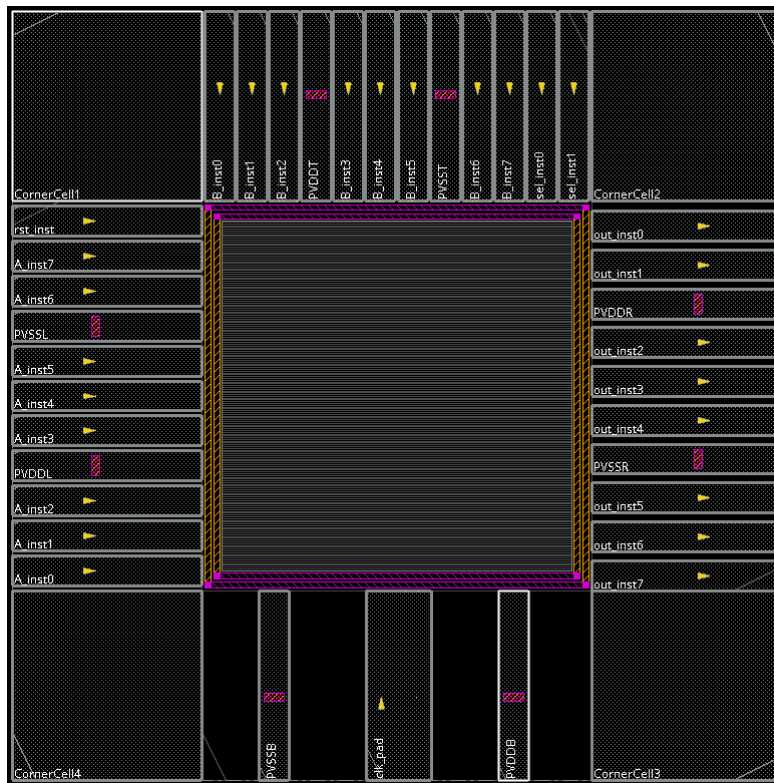


Figure 21: Layout after Power Planning

Make sure that you should place power rings near the **I/O Pads** or between the **I/O Pads and Core** to avoid power losses.

4. Special Route & Power Stripes

In this case, you need to use **\$Route 2 times**. The commands are given below:

```
# --- 4.2 Power Routing (Follow Pins to Core/Pad Rings) ---
set_db route_special_via_connect_to_shape { stripe }

# Route standard cell pins to the nearest available power structure
route_special -nets { VDD VSS } \
  -connect {block_pin core_pin floating_stripe} \
  -layer_change_range { M1(1) M9(9) } \
  -block_pin_target nearest_target \
  -core_pin_target first_after_row_end \
  -floating_stripe_target {block_ring pad_ring ring stripe ring_pin block_pin followpin} \
  -core_pin_check_stdcell_geometry \
  -allow_jogging 1 \
  -crossover_via_layer_range { M1(1) M9(9) } \
  -allow_layer_change 1 \
  -block_pin_use_lef \
  -target_via_layer_range { M1(1) M9(9) }

# Route the pad pins to the pad ring
route_special -nets { VDD VSS } \
  -connect {pad_pin pad_ring floating_stripe} \
  -layer_change_range { M4(4) M9(9) } \
  -block_pin_target nearest_target \
  -pad_pin_port_connect {all_port one_geom} \
  -pad_pin_target nearest_target \
  -floating_stripe_target {block_ring pad_ring ring stripe ring_pin block_pin followpin} \
  -core_pin_check_stdcell_geometry \
  -allow_jogging 1 \
  -crossover_via_layer_range { M4(4) M9(9) } \
  -allow_layer_change 1 \
  -target_via_layer_range { M4(4) M9(9) }
```

Figure 25: \$Route Commands



The layout is look like this which is shown below:

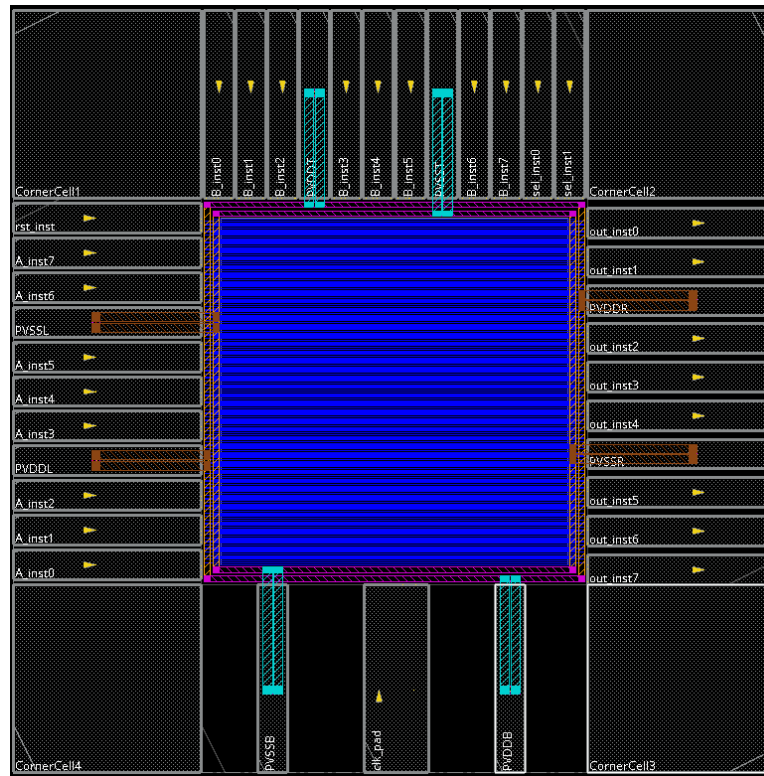


Figure 22: Layout after Special Route

After power stripes commands as you have used before, the output is given below:

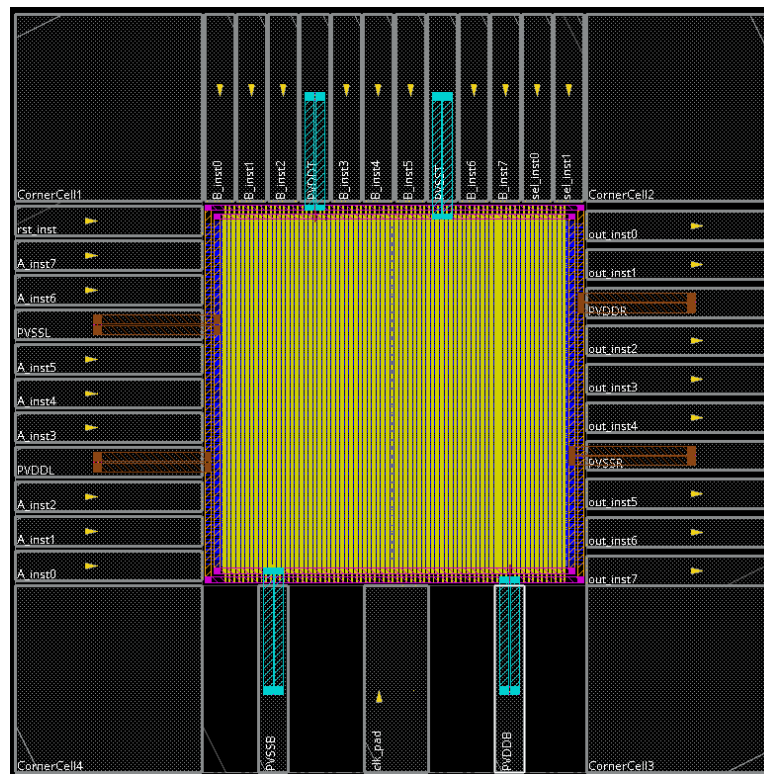


Figure 27: Layout after Special Route



5. Placement of Standard Cell

By giving the parameters you have done in your previous labs, the layout is look like this which is shown below:

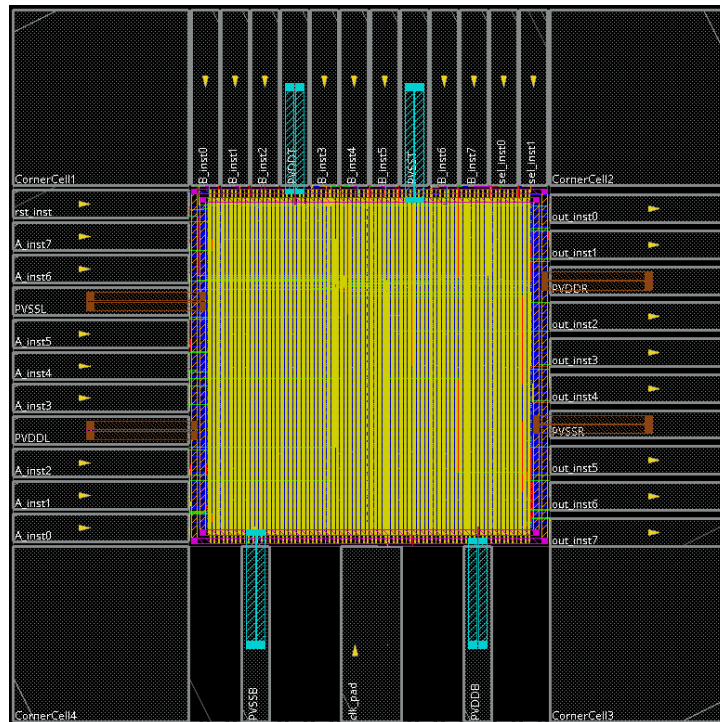


Figure 28: Layout after Placement

6. Complete Process (CTS, Routing, STA, Adding Fillers & DRC)

After going through these processes which you have done in your previous labs, the final layout is look like this which is shown below:

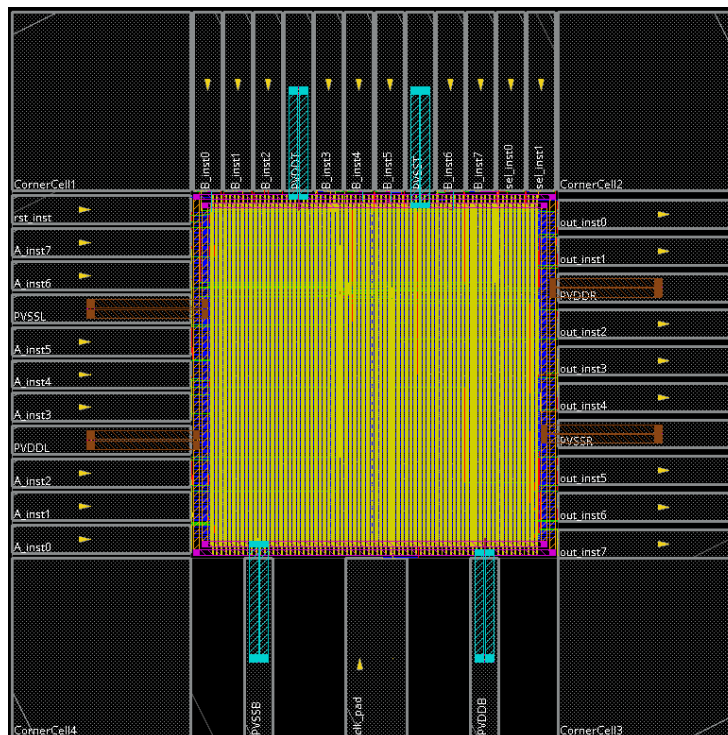


Figure 29: Final Layout



Make sure that there are no violations in DRC & Connectivity Check. Also, you should check timing violation.

10. Lab Task

Given the RTL design of the 8-Bit ALU, do the following:

- i. Read your .lef file & its datasheet carefully.
- ii. Based on these files, write Top Module (Top_Wrapper.v) that instantiate with ALU & I/O Pads Cells.
- iii. Simulate this design using Xcelium & verify its functionality.
- iv. Write constraint file (.sdc) for this scenario.
- v. Write I/O file of your design.
- vi. Synthesize your design using Genus & check all the reports (make sure that there are no timing violations).
- vii. Layout your synthesized design using Innovus & do all the processes which explained at above. Make sure that there are no violations in DRC & Connectivity Check.
- viii. Write detailed report for this lab, make sure that all the process should have written & explain each output.

11. References

- i. [IO pad placement | .io File Writing](#)
- ii. [DIC Flow using Cadence tools By Saurabh Dhiman](#)
- iii. [Digital I/O Circuits - I by Adi Teman](#)
- iv. [Digital I/O Circuits - II by Adi Teman](#)

